



Google Summer of Code 2026



Proposal

Add more timeseries database clients to OpenWISP Monitoring

By : Pushpit Kamboj

Contents

1. Your Details
2. Tell Us About Yourself
3. Your GSoC Project
4. After GSoC

1. Your Details

- **Full Name:** Pushpit Kamboj
- **Date of birth:** 06th July 2003
- **Country/Region:** India
- **Email:** pushpitkamboj@gmail.com
- **Github profile:** [pushpitkamboj](#)
- **Phone number:** xxx (WhatsApp Number)
- **What's your availability in UTC times?**
UTC 3:30 AM to 9.30 AM (25-35 hours per week)

2. Tell Us About Yourself

- **What is your background?**

Hi, I am Pushpit Kamboj and I am currently in my 4th year of Bachelors in engineering (B.E) pursuing computer engineering from Thapar University.

AI enablement intern (Joveo): I am working as an AI enablement intern at a startup named Joveo. I have been working here since January and it will end in July. My major focus is building agentic AI products for the business teams to automate the manual processes. I have worked on different technologies which include Django, OpenAI agents SDK, Gemini SDK, postgres database etc. I also help in guiding the business teams about using AI products for better efficiency. Since I am an intern and the internship is also remote I will be easily able to dedicate my time for GSOC project.

- **Have you ever contributed to open-source software projects? If yes, how?**

Yes, I have contributed to open source before through solving GitHub issues, reviewing PRs alongside maintainers, and guiding new contributors through the contribution process.

My main and regular contribution period started with OpenWISP, where I have raised 6 PRs across different repositories including monitoring module, utils, and controller module, with 4 already merged and 2 open. Beyond OpenWISP, I have also contributed to mesa-llm (mesa org), Augur (CHAOSS org), www-v2 (Sugarlabs org), and Yosemite-Crew (open source startup)

Throughout this period I gained experience in open-source conventions, code review practices, and how larger organizations handle testing and quality assurance for shipped modules. I was also actively involved in the CHAOSS org devs channel, participating in discussions on other contributors' PRs with maintainers going beyond just raising PRs.

Please list the links to your OpenWISP contributions and/or notable contributions to other Open Source & Free Software projects.

OpenWisp Contributions

Description		Status
1. [feature] Added support for non-python packages #564 Repo: openwisp-utils	Merged	
2. [feature] Added changelog entry bot using Gemini AI #584 Repo: openwisp-utils	Merged	
3. [ci:fix] Resolved issues with changelog bot for fork PRs #632 Repo: openwisp-utils	Merged	
4. [bug] Fixed influxDB image by using emulation approach for ARM64 native machines #736 Repo: openwisp-monitoring	Merged	
5. [feature] Added OpenWispPagination class with suitable default values #587 Repo: openwisp-utils	Open	
6. [docs] created docs for websocket API #1225 Repo: openwisp-controller	Open	

Other OSS Contributions

Description	Status
8. <u>Implemented API documentation using Sphinx</u> #46 Repo: mesa/mesa-llm	Merged
9. <u>(fix): remove no else raise and no else return rules from .pylintrc</u> #3533 Repo: augurlabs/augur	Merged
10. <u>Fix types of api admin</u> #472 Repo: Yoshemite-Crew	Merged
11. <u>fix(api): fix syntax to ESM</u> #470 Repo: Yoshemite-Crew	Merged
12. <u>fix(api): add typescript types to ChatMessage model</u> #460 Repo: Yoshemite-Crew	Merged
13. <u>fix(api): remove breeder-details.ts file</u> #456 Repo: Yoshemite-Crew	Merged

Do you have any experience with OpenWrt?

I do not have the OpenWrt experience but since I am interested in hardware and networking I would like to explore it.

Do you have a router at home on which you can flash OpenWrt to test OpenWISP?

I do not have router at home, but I am looking to buy it and then I would try to flash openWrt to test OpenWISP. I am curious to learn about it.

What's your motivation for working on OpenWISP during the Google Summer of Code?

I was exploring the world of open source by trying different tools, libraries, and reading source code treating it less as a checklist and more as a way to understand how serious software is actually built. OpenWISP stood out because it sat at the intersection of Python and real networking infrastructure, two areas I was already drawn to independently.

What made me stay was realizing that universities and municipalities depend on OpenWISP to manage actual network infrastructure, and that weight is visible in how the codebase is structured, how PRs are reviewed, and how maintainers think about changes. Every contribution had to meet a real standard, and that pressure sharpened how I write and reason about code.

3. Your GSoC Project

- **Project Title**

Add more timeseries database clients to OpenWISP Monitoring

- **Possible Mentor**

Federico Capovano, Clément Beaujoin

- **Measurable Outcomes**

1. **Fully working Elasticsearch backend** implementing the complete DatabaseClient interface with all existing tests passing, plus new dedicated tests covering charts introduced after the original ES PR, including signal strength, iperf3, and related metrics.
 2. **Fully working InfluxDB 2.0 backend** implementing the same DatabaseClient interface, with Flux query equivalents written for all 19 chart queries defined in queries.py, including default_chart_query and device_data_query, ensuring full feature parity with the existing InfluxDB 1.8.
 3. **Dedicated GitHub Actions CI pipeline** for each new backend, configured to run the full test suite on every pull request, ensuring regressions are caught before merge and both backends remain green throughout the project.
 4. **Runtime backend switching** via the TIMESERIES_DATABASE setting, allowing operators to choose between InfluxDB 1.8, InfluxDB 2.0, and Elasticsearch without code changes, with installation support integrated into both ansible-openwisp2 and docker-openwisp.
-

5. **Deployment integration** for both new backends in **ansible-openwisp2** and **docker-openwisp**, allowing operators to provision and switch between InfluxDB 1.8, InfluxDB 2.0, and Elasticsearch via a single environment variable with no manual installation steps required.

PROJECT DETAILS

After reading through the OpenWISP Monitoring codebase, I noticed that all database interaction is already isolated behind a single DatabaseClient class. The rest of the monitoring logic never touches InfluxDB directly. It calls methods on timeseries_db, which is a DatabaseClient instance loaded at startup via import_module() based on the settings written in the file. This means adding a new backend is a matter of implementing the same DatabaseClient interface in a new module under openwisp_monitoring/db/backends/. The monitoring logic stays untouched.

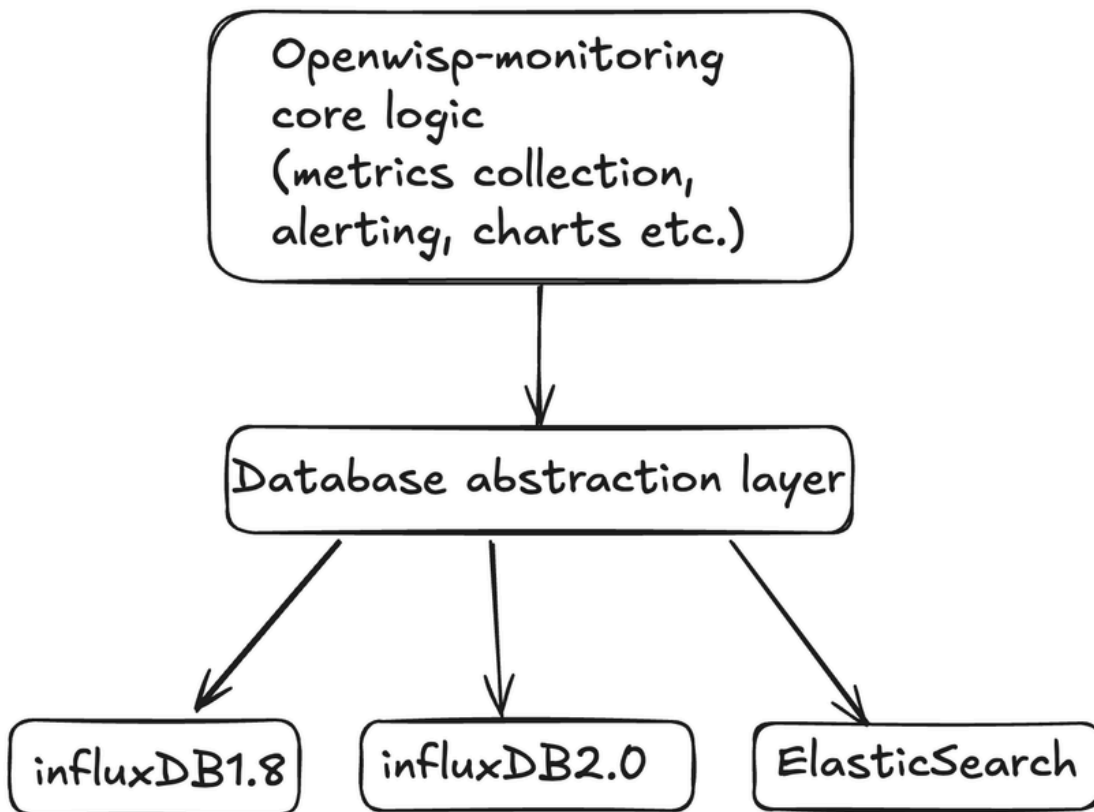
To showcase how I will do this practically I have created a prototype on top of openwisp-monitoring where I have added the support for influxDB2.0 along with test cases and also some basic Flux queries.

PROTOTYPE: https://github.com/pushpitkamboj/openwisp-monitoring/tree/gsoc_prototype/openwisp_monitoring/db/backends/influxdb2

specific files:

client.py queries.py tests.py readme.md

TECHNICAL DIAGRAM



As demonstrated in the diagram above, the core monitoring logic sits at the top, completely decoupled from any database specific code. It interacts only with the abstraction layer, which at runtime loads the appropriate backend based on a single configuration variable. Each backend InfluxDB 1.8, InfluxDB 2.0, or Elasticsearch implements the same interface independently, without the core logic ever needing to know which one is active.

- **How are you going to implement the solution?**

I will explain the solution in 2 parts adding influxDB2.0 and elasticsearch respectively.

InfluxDB 2.0

(All screenshots are taken from the prototype i built. I have referred influxDB1.8 as v1 and influxDB2.0 as v2 in shorthand)

Initialization - v2 requires url, org, token for authentication and db_name while creating the instance.

```
def __init__(self, db_name=None):
    """Initialize InfluxDB 2 client.

    Args:
        db_name: Bucket name (equivalent to database in InfluxDB 1.x)
    """
    self._db = None
    self.db_name = db_name or TIMESERIES_DB["NAME"]
    self.org = TIMESERIES_DB.get("ORG", "openwisp")
    self.token = TIMESERIES_DB.get("TOKEN", "")
    try:
        from influxdb_client import InfluxDBClient
        from influxdb_client.client.exceptions import InfluxDBError

        self.InfluxDBClient = InfluxDBClient
        self.client_error = InfluxDBError
    except ImportError:
        raise ImportError(
            "influxdb-client is required for InfluxDB 2.x support. "
            "Install it with: pip install influxdb-client"
        )
```

create_database() - v2 has a concept of buckets instead of databases and uses buckets_api() for implementation.

```
@retry
def create_database(self):
    """Creates bucket if necessary."""
    api = self.db.buckets_api()
    try:
        bucket = api.find_bucket_by_name(self.db_name)
        if bucket:
            logger.debug(f'InfluxDB2 bucket "{self.db_name}" already exists')
    except self.client_error:
        api.create_bucket(bucket_name=self.db_name, org=self.org)
        logger.debug(f'Created InfluxDB2 bucket "{self.db_name}"')
```

`drop_database()` - this also comes from `buckets_api()`.

```
@retry
def drop_database(self):
    """Drops bucket if it exists."""
    api = self.db.buckets_api()
    try:
        bucket = api.find_bucket_by_name(self.db_name)
        if bucket:
            api.delete_bucket(bucket)
            logger.debug(f'Dropped InfluxDB2 bucket "{self.db_name}"')
    except self.client_error:
        logger.debug(f'InfluxDB2 bucket "{self.db_name}" not found')
```

`retention_policy()`. - significant changes: In v1 we could add multiple retention policies within 1 database but in v2 only 1 retention policy can exist at the bucket(database) level. Also in v2 there is no concept of named policies, means policies created do not require to have a name.

```
@retry
def create_or_alter_retention_policy(self, name, duration):
    """Creates or alters retention policy on a bucket."""
    from influxdb_client.domain import BucketRetentionRules
    api = self.db.buckets_api()
    try:
        bucket = api.find_bucket_by_name(self.db_name)
    except self.client_error:
        logger.warning(
            f'Bucket "{self.db_name}" not found. Create database first.'
        )
        return
    duration_seconds = self._duration_to_seconds(duration)
    bucket.retention_rules = [BucketRetentionRules(
        type="expire", every_seconds=duration_seconds
    )]

    api.update_bucket(bucket=bucket)
    logger.debug(
        f'Created/updated InfluxDB2 retention policy "{name}" with duration {duration}'
    )
```

write() - write_api() is used, v1 supports multi database and multi retention policy routing; v2 writes to a single bucket. I have logged the unsupported parameters as warnings in the prototype.

```
def write(self, name, values, **kwargs):
    timestamp = self._get_timestamp(timestamp=kwargs.get("timestamp"))
    if kwargs.get("database") and kwargs.get("database") != self.db_name:
        logger.warning(
            f'Parameter "database" is ignored in InfluxDB 2.x. '
            f'Using bucket "{self.db_name}"'
        )
    if kwargs.get("retention_policy"):
        logger.warning(
            f'Parameter "retention_policy" is managed at bucket level in InfluxDB 2.x'
        )
    point = {
        "measurement": name,
        "tags": kwargs.get("tags", {}),
        "fields": values,
        "time": timestamp,
    }
    try:
        self._write_api.write(
            bucket=self.db_name, org=self.org, record=point
        )
    except Exception as exception:
        logger.warning(f"Error writing to InfluxDB2: {exception}")
        raise TimeseriesWriteException
```

batch_write() - similar to write() method, batch_write() also logs unsupported parameters, it stores data by write_api().

Due to code being longer i have not pasted the screenshot, I have implemented this method in my PROTOTYPE

query() - significant changes: v1 returns grouped ResultSet objects with get_points(), keys(), items() methods and these methods are used at many different places in the main codebase whereas v2 i.e query_api() returns flat record lists. Now there are 2 choices to have same signature for v1 and v2, either change the existing codebase where instead of returning resultset we return list of dict and remove get_points(), keys(), items() methods OR make v2 backward compatible to mimic and return resultset and this way we don't have to change existing codebase. I have implemented the later solution in the prototype as it felt like a right architectural decision to me.

implementation code: PROTOTYPE

read() - significant changes: v1 implements full InfluxQL query builder supporting DISTINCT, COUNT, WHERE clauses, precision, retention policies via keyword arguments. v2 cannot directly map these parameters to Flux query language syntax. Rather than attempting for translation, v2's read() raises NotImplementedError for unsupported parameters with migration guidance. Basic operations (key, fields, tags filtering) are supported via simple Flux query building. It is advised to use the query() method for complex flux queries. (also implemented this in my prototype)

implementation code: PROTOTYPE

delete_metric_data(): v1 uses DROP SERIES whereas v2 uses delete_api() with predicates. Same parameters (key, tags), similar functionality.

implementation code: PROTOTYPE

`get_list_query()`: significant changes: `query()` method in v1 which returns resultset inherently groups by (measurement, tags) and automatically flattens into time indexed structures. InfluxDB 2.0 `query_api()` method returns a list of FluxTable containing records which require manual grouping. Rather than changing all chart rendering code, we implemented `get_list_query()` as a post-processing layer that iterates through `QueryResultSet` groups, merges tag specific values into single time buckets using tag suffixes, and returns the same time indexed format v1 provides. This centralizes the complexity in one method, avoiding scattered refactoring across the UI layer.

implementation code: PROTOTYPE

`get_list_retention_policies()`: no breaking change, implementation is straightforward through `buckets api`.

```
@retry
def get_list_retention_policies(self):
    """Returns list of retention policies for the bucket."""
    try:
        api = self.db.buckets_api()
        bucket = api.find_bucket_by_name(self.db_name)
        policies = []
        if bucket.retention_rules:
            for rule in bucket.retention_rules:
                policies.append({
                    "name": "default",
                    "duration": f"{rule.every_seconds}s",
                    "replication": 1,
                })
        return policies
    except self.client_error:
        return []
```

get_query(): v1 uses inline InfluxQL string formatting; v2 builds dynamic Flux pipelines by sequentially consuming all parameters (end_date, key, content_type, object_id, ifname, organization_id, location_id, floorplan_id, field_name) as filter stages. Each parameter adds a **|> filter(...)** pipeline, ensuring correct data subset before chart type specific aggregations (wifi_clients COUNT(DISTINCT), traffic GB conversion, rtt mean). This centralizes query logic, making parameter handling explicit and preventing silent data loss.

implementation code: PROTOTYPE

I have discussed and showcased all the methods needed to build influxDB2.0 which is backward compatible with existing InfluxDB1.8.

Apart from methods I have also implemented few queries to demonstrate. InfluxDB2.0 works with flux query language.

query examples: PROTOTYPE

```
chart_query = {
  "uptime": {
    "influxdb2": (
      'from(bucket: "{bucket}")'
      '|> range(start: {time_start})'
      '|> filter(fn: (r) => r._measurement == "{key}")'
      '|> filter(fn: (r) => r.content_type == "{content_type}")'
      '|> filter(fn: (r) => r.object_id == "{object_id}")'
      '|> filter(fn: (r) => r._field == "{field_name}")'
      '|> aggregateWindow(every: 1d, fn: mean)'
      '|> map(fn: (r) => ({r with _value: r._value * 100}))'
    )
  },
  "packet_loss": {
    "influxdb2": (
      'from(bucket: "{bucket}")'
      '|> range(start: {time_start})'
      '|> filter(fn: (r) => r._measurement == "{key}")'
      '|> filter(fn: (r) => r.content_type == "{content_type}")'
      '|> filter(fn: (r) => r.object_id == "{object_id}")'
      '|> filter(fn: (r) => r._field == "loss")'
      '|> aggregateWindow(every: 1d, fn: mean)'
    )
  },
}
```

ELASTICSEARCH 9.3.0

I have read the PR for elasticsearch implementation and noticed that it will require significantly more effort to start working from that than taking inspiration from the code in it and starting fresh. (but i am open to work on existing PR if required)

Similar to InfluxDB2.0 I have explained how each method can be implemented with elasticsearch.

9.3.0 is the latest version. Also attaching the documentation for quick reference [here](#)

initialization: accepts a single host/port and connects to a specific database. ES requires host list (multi-node support), API key auth, and scheme. Index selection happens at query-time, not client creation. Index name is equivalent to database name in influxDB1.8

create_database(): ES creates indices by `indices.create()` with explicit field mappings (@timestamp as date, measurement as keyword, tags/fields as objects) and settings (1 shard, 0 replicas)

delete_database(): ES drops index via `indices.delete()`. Both are single destructive operations. ES ILM policies remain independent and aren't deleted with the index. We check index existence before deletion to avoid exceptions. same method signature and behavior as influxDB1.8

retention_policy(): ES uses Index Lifecycle Management (ILM) policies are separate resources applied to indices via settings.

write(): ES uses bulk API directly. We log warnings for unsupported database/retention_policy parameters for backward compatibility.

batch_write(): v1 groups points by (database, retention_policy) pairs, then executes separate write() per group for multi-database routing. ES batches all points to a single index via bulk API operations. influxDB1.7 scales writes across multiple retention policies; ES scales via bulk operation batching. Same parameter warnings as write() for backward compatibility.

query(): ES executes Query DSL (JSON-based) returning flat hit documents. Instead of refactoring all calling code we can mimic the return type as resultset (we discussed this point above in influxDB2.0). So this way we don't have to change the existing codebase.

read(): v1 builds complex InfluxQL supporting DISTINCT, COUNT, WHERE clauses via keyword arguments. ES doesn't support these parameters directly. We raise NotImplementedError for unsupported params (distinct_fields, count_fields, where, precision, retention_policy) with migration guidance. Basic operations (key, fields, tags filtering) work via Query DSL must clauses. Logs will suggest using the query() method for advanced queries.

delete_metric_data(): ES uses delete_by_query() API with Query DSL predicates. It's straightforward and has no breaking changes.

get_list_query(): Executes search via search(), which returns Elasticsearch hits. And similar to implementation for influxDB2.0, will wrap this into ElasticsearchResultSet to group by (measurement, tags). Iterates through ResultSet.items() to get (key, docs_generator) pairs. So this way all three methods are compatible and the main codebase will be db agnostic.

get_list_retention_policies(): Retrieves ILM policies applied to the index via indices.get(index=self.index) to extract lifecycle policy name from settings, fetches policy details via ilm.get_lifecycle(name=policy_name), and returns list of policy objects with name and duration. Very different from influxDB both v1 and v2 but providing identical method signatures is straightforward.

get_query(): It builds ES Query DSL by sequentially consuming parameters (end_date, key, content_type, object_id, organization_id, location_id, floorplan_id, field_name) as must clauses. Each parameter adds a filter ensuring correct data subset before chart type aggregations (cardinality for wifi_clients, sum for traffic, avg for rtt). Returns Query DSL dict (not JSON string) for consistency with query() method consumption pattern. Centralizes query logic with explicit parameter consumption, preventing silent data loss.

Docker-openwisp and Ansible-openwisp

After the core backend implementations are complete and tested, the next step is ensuring that end users can actually deploy and use them without friction. This involves updating ansible-openwisp2 to add installation and configuration roles for both InfluxDB 2.0 and Elasticsearch 9.3.0, so operators can provision either backend the same way they currently do with InfluxDB 1.8. The same will be done for docker-openwisp, where the compose setup will be extended to include service definitions and environment variable mappings for both new backends.

The goal is that switching between InfluxDB 1.8, InfluxDB 2.0, and Elasticsearch should require nothing more than changing a single configuration variable, no manual installation steps, no custom scripting. Sane defaults will be set for each backend so a basic setup works out of the box, while still allowing advanced configuration for production deployments.

DOCUMENTATION PLAN

Documentation will be written at two levels. For operators, it will cover installation, configuration, and switching between backends via ansible and docker, including backend specific options. For developers, it will document the DatabaseClient interface contract and how to implement a new backend, making future extensions straightforward.

TESTING

Testing will follow the same standards as the existing InfluxDB 1.8 test suite. All current tests must pass against both new backends without any decrease in coverage. New tests will be added specifically for the InfluxDB 2.0 backend covering the new chart types introduced in issue [#270](#), including mobile signal strength metrics for LTE, 5G, UMTS, and GSM. Each backend will have a dedicated CI build that runs on every pull request, ensuring regressions are caught early and the backends do not silently break as the codebase evolves.

For Elasticsearch specifically, query optimization will be implemented as outlined in issue [#168](#) this means structuring requests to minimize round trips, using bulk APIs where applicable, and ensuring aggregation queries are efficient at scale rather than naive one-to-one translations of the InfluxQL originals. The dedicated Elasticsearch CI build will act as the continuous gate to validate both correctness and that the optimization constraints are maintained across future changes.

Addressing open questions

I am open to adding TimescaleDB support. Since OpenWISP already uses PostgreSQL as its primary database, operators running smaller deployments would not need to install or maintain a separate time series database at all. Metrics could live in the same PostgreSQL instance that already stores device configurations, users, and organizations. This eliminates the need for a separate TSDB process entirely, reducing operational overhead for smaller deployments

What technologies do you want to use

- Languages: Python, Flux query language, Elasticsearch Query DSL
- Database clients: influxdb-client, elasticsearch
- Testing: Django test framework, pytest
- CI/CD: GitHub Actions
- Deployment: Ansible (ansible-openwisp), docker (docker-openwisp)

Prototype and proof of concept

I have implemented InfluxDB2.0 on openwisp-monitoring repo showcasing how I will add it during my gsoc timeperiod. Along with that i have also written 6 Flux queries and also written the test cases. Here I am attaching the links of each of the file i have written

[client.py](#). [queries.py](#). [tests.py](#). [readme.md](#)

Availability

I can commit 25-35 hours per week to the project. I am currently doing a remote internship and it will end in July, I work there 20-25hours per week. Internship being remote eliminates commute time and gives me a flexible daily schedule to accommodate consistent contributions throughout the program. I have no exams or travel planned during the GSoC period.

Project Schedule

Week	Focus	Deliverable
Pre-GSoC	Early start, environment setup	Dev environment ready, all three database services running locally, existing test suite passing
Week 1-3	InfluxDB 2.0 core backend	Implement all core data methods using influxdb-client library
Week 4-5	InfluxDB 2.0 Flux query migration	Translate all 19 chart queries from InfluxQL to Flux, implement chart query methods for Flux
Week 6-7	InfluxDB 2.0 PR review, testing and CI	All existing InfluxDB 1.8 tests replicated and passing on the 2.0 backend, dedicated CI build configured
Week 8-11	Elasticsearch core backend	Implement all core data methods using elasticsearch library, apply optimizations from #164
Week 12	Mid term evaluation	Get feedback from the maintainer about the above deliverables
Week 13-15	Elasticsearch query layer	Translate all 19 chart queries from InfluxQL to elasticsearchDSL,implement chart query methods
Week 16-18	ElasticSearch PR review, testing and CI	All existing InfluxDB 1.8 tests replicated and passing on the elasticsearch backend, dedicated CI build configured
Week 19-20	ansible-openwisp2 and docker-openwisp	Add choosing between all 3 backends for better UX.
Week 21-22	Documentation	Dedicated page for documenting the usage for both the backends for both users and developers.
Week 22+	timescaleDB	If the work finishes on time, I would be happy to extend the support to timescaleDB

After GSOC

Are you interested in continuing to collaborate with OpenWISP after the GSoC ends?

Yes. OpenWISP is a production grade project used in real network infrastructure, and contributing to it has been a genuinely different experience from anything I have built on my own. Apart from the GSOC project I am also very interested in task scheduling problems as it involve core backend engineering challenges that force first principles thinking, and I would love to explore them deeper within OpenWISP once I get more hands on with the codebase. If time permits after core deliverables, I would also like to explore adding TimescaleDB as a fourth backend, since OpenWISP already uses PostgreSQL.

Will you help maintain your implementation for a while?

Yes. I want to see both backends stable and actively used in production, not just merged and forgotten. I will stick around to address bug reports, handle compatibility issues with newer versions of InfluxDB and Elasticsearch, and review any follow-up contributions related to my implementation

If we get new business opportunities to build new features, are you interested in occasional freelance paid work?

Yes, absolutely. By the time GSoC ends I hope to have developed a solid understanding of the OpenWISP monitoring internals and how everything fits together. That kind of depth takes time to build and I would genuinely love to keep contributing and applying it on meaningful work. If there are opportunities to do that through occasional paid engagements, I would be very happy to be considered.